magic-r

R listings that link to their documentation, with flowR and R

Florian Sihler*

Version v1.0 – 2026/10/10

github.com/EagleoutIce/magic-r

1. Introduction 1
2. Getting Started 2
3. Linking without flowR 3
4. Analyzing with flowR 6
5. Computing with R 13
6. Known Limitations 14
7. Benchmark 14
A. The Shipped Database 15
Index of Commands, Options, and Keys 16

1. Introduction

This package allows you to write plain R in your documents and slides, and get listings that link every known name to its help page, in R's own manual or on rdrr.io. Everything you define links back to its definition. Packages attach at the points where the code loads them, so after `library(dplyr)` the name `filter` means `dplyr::filter`, and before it `stats::filter`. Everything builds on `xlistings` and works with its styles, environments, inline commands, and `minted` emulation. The package is the sibling of `magic-haskell` for R.

```r
library(dplyr)

#' The mean consumption per number of cylinders
#' @param data a data frame like mtcars
consumption ⟵ function(data, min_cyl = 4) {
  data ▷
    filter(cyl ≥ min_cyl) %>%
    group_by(cyl) ▷
    summarise(mpg = mean(mpg), n = n())
}
consumption(mtcars)
```

In this listing, the functions of dplyr and R link to their help pages, `consumption` and its parameters link to their definitions, and the named arguments (`mpg = ...`) do not link. The roxygen comment shows its tags, and the operators are symbols that copy as <- and |>. R can also compute values for the text: `\mrValue{round(mean(mtcars$mpg), 2)}` gives 20.09, which took R 9 ms.

Each feature in this manual has a badge that says what it needs:

- always works with nothing installed: the shipped database of 101 packages, R's search path, definitions, roxygen, ligatures, and markers (Section 3).
- flowR needs `flowR`, a static analyzer for R (Section 4). The badges node docker r web name the backends of flowR a feature works with (Section 4.1).
- R needs R, which computes values and console sessions (Section 5).

*Released under the LaTeX Project Public License, version 1.3c or later.

2. Getting Started

```
\usepackage[eval]{magic-r} % 'eval': ask R for values and sessions
\usepackage{hyperref}   % links need hyperref, load it after magic-r
...
\begin{mr}
x <- paste("a", 1:3)
\end{mr}
```

The language is called `mr`, so there is an environment `mr`, the inline command `\bmr`, and `\begin{minted}{mr}` (see `xlistings` for all of them). flowR and R need `-shell-escape` and only run when their input changed. Without it, the package warns, prints the command to run by hand, and the answers show **??**. A copy of the package in a folder of the project needs no change of `TEXINPUTS`: with `\usepackage{_config/magic-r/magic-r}` it finds its database and its script beside it.

2.1. Package Options

- **`language`** (default `mr`) the name of the language, `language=R` replaces the R of `xlistings` (a later `\LoadLanguage{R}` does not, magic-r warns and registers its own again). A name whose inline commands exist already is an error, `language=r` would turn `\cr` into inline code and break every table, so magic-r takes `mr` instead. For minted, `R` and `r` are aliases.
- **`links`** (default `true`), **`underline`** (default `true`) link names, and underline the links slightly (in the color `mrlink`). `\mrLinksOn`, `\mrLinksOff`, `\mrUnderlineOn`, and `\mrUnderlineOff` switch them anywhere, the key `mrlinks=false` for one listing.
- **`locallinks`** (default `true`) link names to their definitions in the document (Section 3.3). `\mrLocalLinksOn` and `\mrLocalLinksOff` switch them anywhere, the key `mrlocallinks=false` for one listing.
- **`ligatures`** (default `true`), **`tight`** (default `true`) typeset operators such as `<-` with their characters moved closer together, **`symbols`** (default `false`) replaces them by symbols, such as an arrow for `<-` (Section 3.9). `\mrLigaturesOn`, `\mrLigaturesOff`, `\mrTightOn`, and `\mrTightOff` switch them.
- **`operators`** (default `false`) link operators and keywords to R's help as well: `<-` to `assignOps`, `|>` to `pipeOp`, `if` to `Control`, `::` to `ns-dblcolon`. They keep their color and get no underline, so the code does not look busier than without links.
- **`strict`** (default `false`) link only names of attached packages. Otherwise a name of a package the document uses links even before it is attached.
- **`database`** (default `magic-r-links`) the file of the database (without `.cfg`).
- **`flowr`** (default `auto`) when flowR runs: `auto` with unrestricted shell escape, `always`, `never` (it prints the command to run by hand), or `off`, which also stops writing the code.
- **`flowr-backend`** (no default) the backend of flowR to use, e.g. `node` or `docker` (Section 4.1). By default the first one that works.
- **`flowr-engine`** (default `lax`) how flowR parses: `lax` is tree-sitter with its lax parser, which reads code with syntax errors, so a fragment in one listing does not hide the others. `tree-sitter`, `r-shell` (needs R), and `default` are the others.
- **`flowr-command`** (no default) the flowR command line to run, e.g. of a local build, **`flowr-image`** (no default) the image of docker (Section 4.1).
- **`flowr-version`** (no default) the version flowR must have: `2.16.1` exactly, or a bound such as `>=2.15` or `<3`. An exact version switches flowR to that version (Section 4.1).
- **`flowr-required`** (default `false`) turn a missing or failing flowR into an error instead of a warning. For example, `flowr-backend=node, flowr-version={>=2.15}, flowr-required` requires the flowR command line in version 2.15 or later.
- **`index`** (default `none`) index the symbols of every listing, by `usage` or as `ssa` (Section 3.7).
- **`slices`** (default `dim`) gray out (`dim`) or leave out (`hide`) what a slice drops (Section 4.5).
- **`lint`** (default `false`) run flowR's linter on every listing (Section 4.6).

- **dependencies** (default false) mark the lines that read, write, or source files (Section 4.4).
- **graphs** (default false) load TikZ for the graphs of flowR (Section 4.8).
- **graphs-gray** (default false) draw the graphs in shades of gray, for print.
- **graphs-legend** (default false) give graphs a legend by default (Section 4.8).
- **graphs-lines** (default false) show the line of its code in every node of a graph by default (Section 4.8).
- **playground** (default false) a button beside every listing opens it in flowR's playground (Section 4.9).
- **arrowroute** (default auto) the route of arrows: `auto` lets an arrow to a definition above run over the top where it can, `side` keeps every arrow beside the code (Section 4.7).
- **interactive** (no default) load `ocgx2` for listings whose arrows a click shows or hides (key `mrinteractive`), `interactive=all` makes every listing so (Section 4.7).
- **eval** (default false) let R compute values and sessions (Section 5).
- **cleanup** (default true) the script removes the side files of older versions of magic-r, one per listing (`\jobname.mr-play-3.R`), now one file of each kind holds all listings. `cleanup=false` keeps them.
- **xlistings** (no default) options for `xlistings`, e.g. `xlistings={style={plain number}}`. They only count when magic-r loads `xlistings`. A document that loads `xlistings` itself (also by its path) may do so before or after magic-r, magic-r uses it as it is. magic-r needs `xlistings` 2026/08/17 or later and warns about an older one.

3. Linking without flowR

always Needs nothing but LaTeX.

3.1. Linking to the Documentation

Each package of the database lists its names by help topic, and a name links to the page of its topic: R's manual (`\mrDocsRManual`) for R's own packages, rdrr.io (`\mrDocsCran`) for CRAN, and a package name to its CRAN page (`\mrCran`). Redefine these macros to link elsewhere. A package is only read when a document uses it. The database holds all of R's own and recommended packages, the whole tidyverse, and other popular packages, 101 in all (Section A). An S3 method without a page of its own links to its generic: `print.character(x)` and `as.data.frame.matrix(m)` link to `print` and `as.data.frame`. The database knows the generics of each package, and a method the document defines links to its definition.

3.2. Attaching Packages

As in R, the attached packages form a search path, and a name links to the first package on it that exports it. It starts with R's defaults (`\mrDefaultSearchPath`). `library(pkg)` and `require(pkg)` attach a package where they appear, but not `library(p, character.only = TRUE)`, where `p` is a variable. By hand:

- `\mrAttach{pkg,...}`, `\mrDetach{pkg,...}` attach and detach packages. The key `attach` does the same for a listing, `\bmr[attach=dplyr]{n()}`.
- `\mrSearchPath{pkg,...}` sets the whole path, the first package is searched first.
- The key `mrlibrary={pkg,...}` lets a listing run as if it started with `library(pkg)` for each package, without showing these calls. The packages are attached for this listing only, and the code of its playground link (Section 4.9) starts with the calls.
- The key `mrpackage` makes every name of inline code a package name, with the style and link of a package, for names of packages in the text: `\bR[mrpackage]{dplyr}`.
- `\mrLoadPackage{pkg,...}` reads packages without attaching them, so that their names link if no attached package has them (unless `strict`).
- `\mrVersion{pkg}` is the version of a package in the database: `dplyr` unknown.

`pkg::name` and `pkg:::name` always link into the package.

3.3. Linking Your Own Definitions

In a displayed listing, these define `x`:

- an assignment at the start of a line: `x ⟵ 1`, `x ⟸ 1`, `x = 1`,
- an assignment to the right: `1:3 ⟶ x`, `1:3 ⟹ x`,
- `assign("x", 1)`,
- a parameter: `function(x, n = 2)`, for the uses in the body of its function, also if the listing has a variable of the same name.

A use of `x` links to the definition in the same listing, else to the last one before, else to the first one after (not in a listing flowR read: there a use only links to what flowR finds, or to a definition in the same or an earlier listing). In `get("x")`, `get0("x")`, and `exists("x")`, the string links to the definition and keeps its color. Functions (`f ⟵ function(x)`, `f ⟵ \(x)`) look like the functions of packages, variables keep the color of the code, and parameters get the color `mrparam` (a shade of the color of packages, `\colorlet{mrparam}{...}` after `\begin{document}` changes it). An indented assignment only counts within its listing. `\mrSetStyle{kind}{style}` sets the style of a kind of name: `function` (of packages), `user` (functions of the document), `variable`, `parameter`, `package`, `data` (data sets of packages, as `relig_income`, plain as variables by default), `operator`, and `argument` (`n` in `head(x, n = 3)`, plain by default), e.g. `\mrSetStyle{user}{\color{gray}}`. The key `mrdefinitions` switches this off (`false`) or keeps all definitions of a listing local (`local`). Definitions come from the aux file, so they need a second run, as references do.

```
\begin{mr}
square <- function(x) x^2
square(3) + square(4) -> total
assign("half", total / 2)
get("half")
\end{mr}
```

```
square ⟵ function(x) x^2
square(3) + square(4) ⟶ total
assign("half", total / 2)
get("half")
```

Now `total`, `half`, and `square(2)` link to their definitions.

3.4. Names That Never Link

Strings and comments, named arguments (`paste(sep = "")`, found in the previous run), what follows `$` and `@` (`df$mean`), and R's keywords never link.

3.5. Highlighting Roxygen Comments

Comments that start with `#'` are roxygen documentation: their tags (`@param`) and the name after `@param` stand out, and `pkg::name` links, as in `[base::sum()]`.

```
\begin{mr}
#' Scale values to the unit interval, see [base::range()]
#' @param x a numeric vector
#' @return x, scaled
#' @export
unit <- function(x) (x - min(x)) / diff(range(x))
\end{mr}
```

```
#' Scale values to the unit interval, see [base::range()]
#' @param x a numeric vector
#' @return x, scaled
#' @export
unit ⟵ function(x) (x - min(x)) / diff(range(x))
```

3.6. Writing Inline Code

\bmr reads its argument verbatim, so %, #, $, and \ need no escape: \bmr{x %in% y$z} gives x %in% y$z. Within the argument of another command (\textbf{\bmr{...}}) TEX has read the characters already: write \%, \#, and \$ there, and \\ for a backslash.

3.7. Indexing Symbols

always The key mrindex (or the option index for all listings) writes plain R with indices, as the IndexR of xlistings does by hand. mrindex=usage (the default) numbers the uses of each symbol, mrindex=ssa the versions of each variable: a definition makes a new version, and a use has the version of the definition it reads. Without flowR, that is the last definition before its line. The indices do not copy. \mrUse[label]{symbol}{index} typesets a symbol with its index in the text and links to its place in the listing with the key mrname=label, by default the last listing with a name.

```
\begin{mr}[mrindex,mrname=usage]
x <- c(1, 2, 3)
y <- x * 2
x <- x + y
print(mean(x))
\end{mr}
```

```
x₁ ⟵ c₁(1, 2, 3)
y₁ ⟵ x₂ * 2
x₃ ⟵ x₄ + y₂
print₁(mean₁(x₅))
```

\mrUse{x}{4} gives the fourth use of x, x_4.

```
\begin{mr}[mrindex=ssa,mrname=ssa]
x <- c(1, 2, 3)
y <- x * 2
x <- x + y
print(mean(x))
\end{mr}
```

```
x₁ ⟵ c(1, 2, 3)
y₁ ⟵ x₁ * 2
x₂ ⟵ x₁ + y₁
print(mean(x₂))
```

\mrUse{x}{2} gives the second version of x, x_2, and \mrUse[usage]{x}{4} refers to the listing before. In every listing with a name, \mrUse also finds the uses of a symbol without indices. \mrUseAt[label]{criterion} names a place instead, with the index the listing shows: \mrUseAt{3@[2]x} gives x_1, the second x of line 3, \mrUseAt{2@y} the first y of line 2, y_1, and \mrUseAt{3:6} the name at line 3, column 6 (of the code without its indentation), x_1.

3.8. Adding and Blocking Links

\mrAddLink{url}{names} links the names to a page of your choice, ahead of all packages. \mrNoLink{names} stops them from linking. \mrPackage[docs]{pkg}{version}{entries} adds a whole package, like the database: the entries \N{names} link to topics of the same name, \T{topic}{names} to a topic, \U{url}{names} to a page, and docs is rman or cran. \mrOperators{\%op\%, ...} declares %op% operators.

3.9. Typesetting Operators

<-, |>, and the other operators in \mrTightLigatures move their characters closer together by \mrLigatureKern (.05em) but keep their width. With the option symbols (on in this manual), <-, <<-, ->, ->>, |>, <=, >=, !=, &&, and

|| become symbols of the same width, as in `x ⟵ y ▷ f() && z ≠ 0`. Selecting them copies the original text. Operators such as `%in%` stay as they are. `\mrTightLigature{op}` and `\mrLigature{op}{display}` add others.

3.10. Marking Code

`|warn|...|warn|`, `|err|...|err|`, and `|info|...|info|` put a band behind what they enclose, also over several lines, and keep its links. `|plain|...|plain|` and `|dim|...|dim|` typeset code plainly or dimmed, `!*...*!` escapes to LaTeX (not in inline code). flowR reads an escape as its text: `!*\textbf{x}*!` is x.

```
\begin{mr}
y <- |warn|sapply(x, mean)|warn| + |err|undefined_thing|err|
\end{mr}
```

```
y ⟵ sapply(x, mean) + undefined_thing
```

4. Analyzing with flowR

flowR magic-r writes the code of all listings to `\jobname.mr.R`. If it changed, the script `magic-r-flowr.mjs` (needs `node`) asks flowR at the end of the run and writes the answer to `\jobname.mr.cfg` for the next run. If flowR cannot run, a warning says why and Section 3 still applies. The key `mrcapture=false` keeps a listing from flowR.

flowR analyzes the listings of the document as one program, so a definition in one listing reaches all later ones, and so does a listing that rebinds `<-` or `function`. The key `mrscope=name` puts a listing into a scope of its own. flowR analyzes each scope as a separate program, and the listings without a scope form the default one. `\mrScope{name}` sets the scope of all listings that follow, and `\mrScope{}` returns to the default. A name has no spaces.

Files on disk take part in two ways. `\imr[options]{file.R}` (or `\lstinputlisting[language=lMagicR]{file.R}`) shows one as a listing like any other. The key `mrsource={file,...}` uses files without showing them: the listing runs as if it `source()`d them first, names it reads from them get the style of their kind but no link, packages they load count as loaded, and its playground has them as files of their own. TeX finds them through its search path, and a change of one runs flowR again. Slices, arrows, and graphs analyze only the listing.

4.1. Choosing a Backend

magic-r uses the first of these backends that works, or the one the option `flowr-backend` sets:

- `node` the flowR command line: the option `flowr-command`, `$MAGIC_R_FLOWR`, `flowr` on the path, or the flowR of `npm install -g @eagleoutice/flowr` (an empty `$MAGIC_R_FLOWR` turns this backend off). A local build of flowR works as well: `flowr-command={node ~/flowr/dist/src/cli/flowr.js}`.
- `docker` the image of the option `flowr-image` (default `eagleoutice/flowr:latest`), if docker has it.
- `r` the R package `flowr`.
- `web` the signature table of flowR's signature browser, with the loads found by a plain scan of the code. It knows packages, nothing else.

Packages that flowR does not know are read from an installed R. Each package is cached per version (`$MAGIC_R_CACHE`, else the user's cache), the version of each flowR for a day, and flowR's answers by the flowR and the code they are of: only changed listings and requests go to flowR, and a run that changes none does not start it. Answers unused for 30 days are removed.

An exact `flowr-version` (`flowr-version=2.16.1`) picks the docker image `eagleoutice/flowr:2.16.1` if docker has it, else the npm package, installed into the cache once (`\mrFlowrOptions{--no-install}` forbids that, `node magic-r-flowr.mjs install 2.16.1` does it ahead of time). The flowR of `flowr-command` is used if it has that version.

magic-r works with flowR 2.2 and later. Before 2.12 the links of packages come from an installed R and the shipped database, control flow graphs need 2.14 (magic-r warns and leaves them out). From 2.16 on, dataflow graphs come without environments (faster), and flowR runs in its secure mode: no R, no network, and no mermaid links on the clipboard. Newer versions work as long as flowR's queries keep their form.

- `\mrFlowrOptions{options}` passes options to the script, e.g. `\mrFlowrOptions{--timeout 300}` or `--flowr <command>`, see `node magic-r-flowr.mjs --help`.
- `\mrFlowrRun{auto|always|never}` overrides the option `flowr`, `\mrNodeCommand{node}` the command of node.

node docker To compare flowR versions or builds on one document, a listing (key `mrflowr`) or a graph or query (key `flowr`) can name another flowR, as `docker:<image>` or `node:<command>`. That flowR answers the graphs, slices, arrows, findings, and queries of the listing or command, while names still link as the document's flowR says. `\mrFlowrDescription{flowr}` says which flowR answered, `\mrFlowrDescription{}` the one of the document (here: node flowR 2.16.1).

4.2. Finding Packages

node docker r web flowR finds every package the code loads or uses, also the ones the database lacks, and where it is loaded: through `pacman::p_load`, in a loop over `library(p, character.only = TRUE)`, or with `requireNamespace`.

```
\begin{mr}
pkgs <- c("tidyr", "purrr")
for (p in pkgs) library(p, character.only = TRUE)
pivot_longer(relig_income, !religion) |> map(nrow)
\end{mr}
```

```
pkgs ⟵ c("tidyr", "purrr")
for (p in pkgs) library(p, character.only = TRUE)
pivot_longer(relig_income, !religion) ▷ map(nrow)
```

4.3. Linking Names by Dataflow

node docker r flowR's dataflow graph replaces the rules of Section 3.3:

- Every definition counts, wherever it is (`if (ok) y ⟵ 1`, in a loop, in a function), and every use links to the definition it reads, also across listings and in inline code.
- A local variable named like a function of a package (`filter ⟵ 2`) links to its definition, while `filter(df)` still calls the function of the package.
- A backtick name (`` `my var` ``) links as one name.
- Parameters link within their function, named arguments never link, and `library(p, character.only = TRUE)` reads the variable `p`.

flowR parses with tree-sitter's lax parser (option `flowr-engine`), so a fragment such as `f(x,` does not hide the code around it.

4.4. Marking File Access

node docker r With the option `dependencies` or the key `mrdependencies`, lines that read, write, source, plot, or run a statistical test get a band in the colors `mrread`, `mrwrite`, `mrsource`, `mrvisualize`, and `mrstatistics`, and the first line of a run names its kind at the right. Printing is no write.

```
\begin{mr}[mrdependencies,mrname=files]
raw <- read.csv("measurements.csv")
source("helpers.R")
clean <- subset(raw, !is.na(value))
write.csv(clean, "clean.csv")
plot(clean$value, type = "l")
lines(lowess(clean$value), col = "steelblue")
title("Measurements")
t.test(clean$value, mu = 20)
\end{mr}
```

```r
raw ⟵ read.csv("measurements.csv")
source("helpers.R")
clean ⟵ subset(raw, !is.na(value))
write.csv(clean, "clean.csv")
plot(clean$value, type = "l")
lines(lowess(clean$value), col = "steelblue")
title("Measurements")
t.test(clean$value, mu = 20)
```

4.5. Slicing Listings

node docker r The key `mrslice={criterion}` keeps what affects the criterion: a variable in a line (`5@total` for its first occurrence there, `3@[2]v` for the second), or a line and column (`5:1`). The key `mrslicemode` (default: the option `slices`) grays out the other lines (`dim`) or leaves them out (`hide`), with the numbers of the lines kept.

```
\begin{mr}[mrslice={5@total}]
total <- 0
count <- 0
for (v in c(1, 4, 9)) total <- total + v
count <- count + 1
print(total)
\end{mr}
```

```r
total ⟵ 0
count ⟵ 0
for (v in c(1, 4, 9)) total ⟵ total + v
count ⟵ count + 1
print(total)
```

```
\begin{mr}[mrslice={5@total},mrslicemode=hide]
total <- 0
count <- 0
for (v in c(1, 4, 9)) total <- total + v
count <- count + 1
print(total)
\end{mr}
```

```r
total ⟵ 0
for (v in c(1, 4, 9)) total ⟵ total + v
print(total)
```

4.6. Underlining Linter Findings

node docker r The key `mrlint` (or the option `lint` for every listing) runs flowR's linter. A finding gets a wavy line below its code, red if flowR is certain, else orange, and its line names the rules at the right, linked to their description in flowR's wiki. With the option `playground`, a finding opens flowR's playground with the finding marked. `\mrLintShape{wave|dash|line}` chooses the line. `mrlint={["unused-definitions"]}` picks rules (with their configuration, as in flowR's query API), `\mrLinterRules{rules}` sets them for all listings.

```
\begin{mr}[mrlint]
data <- read.csv("/home/me/data.csv")
unused <- nrow(data)
setwd("/tmp")
print(summary(data))
\end{mr}
```

```r
data ⟵ read.csv("/home/me/data.csv")
unused ⟵ nrow(data)
setwd("/tmp")
print(summary(data))
```

file-path-validity, absolute-file-paths
unused-definitions
absolute-file-paths

4.7. Drawing Arrows to Definitions

node docker r With the option graphs, the key mrarrows={criteria} draws an arrow from the name each criterion points at to every definition it reads. Other names get no arrows. An arrow to a definition above leaves its name at the top, runs below the line above, and goes straight up into the definition, if no letter and no other arrow is in its way. Every other arrow runs below the line of the use, along a lane left of the code, and below the line of the definition into it, so it crosses no letters. The key mrarrowroute=side (or the option arrowroute=side) keeps all arrows beside the code. An arrow within one line stays below that line. Arrows that would overlap get their own height below a line and their own lane, and arrows that share a name spread over its width, in the order of the sides they come from. Each criterion (in an interactive listing, each variable) has a color (mrarrow1 to mrarrow5, in order), the names of the criteria get a darker band and the definitions a lighter one. \mrArrowGap, \mrArrowTrack, and \mrArrowLane set the distances, the TikZ style mr arrow the look. A listing with arrows is not broken across pages.

```
\begin{mr}[mrarrows={6@total, 4@v, 5@[2]count}]
total <- 0
count <- 0
for (v in c(1, 4, 9))
  total <- total + v
count <- count + 1
print(total)
\end{mr}
```

```r
total ⟵ 0
count ⟵ 0
for (v in c(1, 4, 9))
  total ⟵ total + v
count ⟵ count + 1
print(total)
```

The key mrcallsarrows={criteria} draws the calls. Where they cannot go over the top, they run along a lane right of the code, apart from the dataflow at the left, or left of the code where no dataflow runs along their lines and that way is shorter (the TikZ style mr call arrow sets their look): from each call a criterion points at to the definition of the function it calls (also through another name, as h ⟵ f). A criterion that points at the name of a function takes all of its calls, and mrcallsarrows=all every call of a function of the listing. Calls of functions of packages get no arrow.

```
\begin{mr}[mrcallsarrows=all]
square <- function(x) x^2
add <- function(a, b) a + b
total <- add(square(3), square(4))
twice <- function(v) add(v, v)
twice(total)
\end{mr}
```

```r
square ⟵ function(x) x^2
add ⟵ function(a, b) a + b
total ⟵ add(square(3), square(4))
twice ⟵ function(v) add(v, v)
twice(total)
```

With the option interactive (it loads ocgx2), the key mrinteractive makes every name of a listing a switch for its arrows, as optional content of the PDF: a click on a use shows or hides the arrows to the definitions it reads, a click on a definition those of the uses that read it. The names the criteria of mrarrows point at show

theirs at first, and `interactive=all` makes every listing interactive. An interactive listing has no links to the definitions of the document, so that a click on a name always shows or hides its arrows (`mrlocallinks=false`, which `mrlocallinks=true` undoes). This works on beamer slides as well, and in viewers that support optional content, such as Acrobat Reader or Okular. Click the names below:

```
\begin{mr}[mrinteractive,mrlocallinks=false,mrarrows={6@total}]
total <- 0
count <- 0
for (v in c(1, 4, 9))
  total <- total + v
count <- count + 1
print(total)
\end{mr}
```

```
total ⟵ 0
count ⟵ 0
for (v in c(1, 4, 9))
  total ⟵ total + v
count ⟵ count + 1
print(total)
```

4.8. Drawing Dataflow and Control Flow Graphs

node docker r With the option `graphs`, `\mrDataflowGraph[keys]{label}` draws the dataflow graph of the listing with the key `mrname=label`, `\mrControlFlowGraph[keys]{label}` its control flow graph in basic blocks, and `\mrCallGraph[keys]{label}` its call graph: which function calls which, from the top level to the functions of R. Their keys:

- `legend=true|false` a legend below the graph that names its shapes and edges (default: the option `graphs-legend`),
- `lines=true|false` the line of its code in every node, after the id of a vertex (`\mrGraphLine` typesets it, default: the option `graphs-lines`),
- `rowsep=cm` the least distance between rows (default `0.2`), rows with edges running sideways get more,
- `mermaid=text` instead of the graph a link to flowR's own drawing of it on mermaid.live (default text `mermaid.live`), made from the current code,
- `function=name` only what a function calls, for the call graph,
- `flowr=flowr` another flowR for this graph (Section 4.1),
- any other key is an option of TikZ, e.g. `scale=.7,transform shape`.

A vertex of the dataflow graph shows its id (`\mrGraphId`) and what it reads or calls of R itself (`\mrGraphBuiltin`). As in flowR's mermaid graphs, definitions are rectangles, uses stadiums, values hexagons, and calls have bars at their sides. In the control flow graph, the entry and the exits are stadiums and conditions hexagons. TikZ styles such as `mr node`, `mr vdef`, `mr fcall`, or `mr reads` change the look.

```
\begin{mr}[mrname=loop]
total <- 0
for (v in c(1, 4, 9)) {
  if (v > 2) total <- total + v
}
print(total)
\end{mr}
```

```
total ⟵ 0
for (v in c(1, 4, 9)) {
  if (v > 2) total ⟵ total + v
}
print(total)
```

```
\mrDataflowGraph[legend,lines]{loop} \hfill
\mrControlFlowGraph[legend,rowsep=0.3]{loop}
```

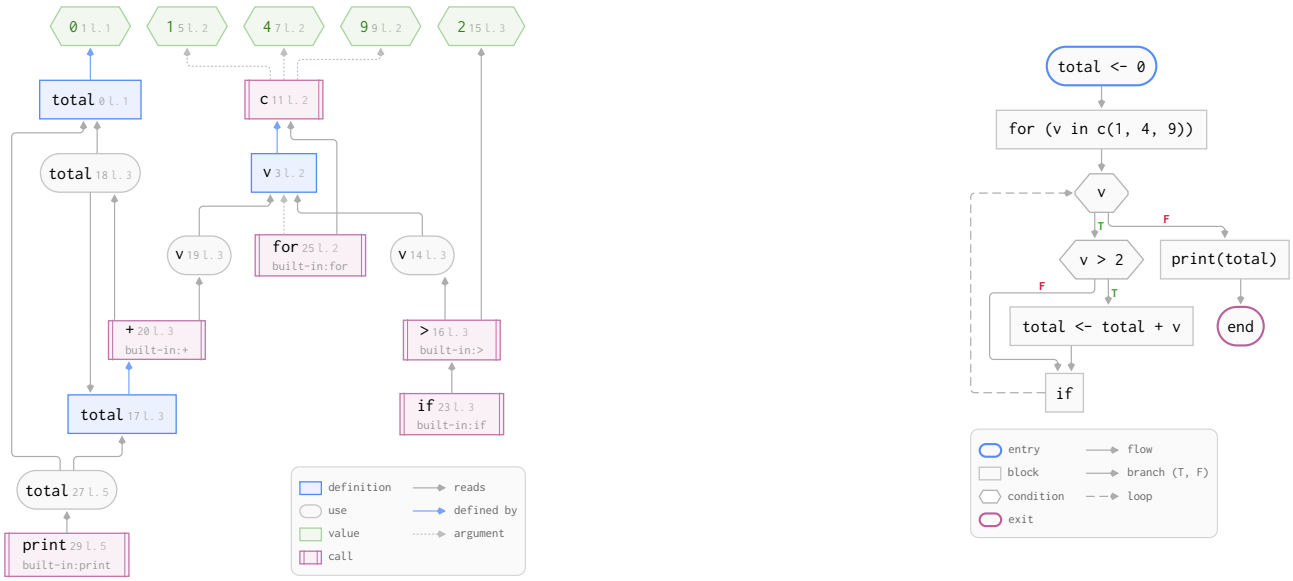


Figure 1: The dataflow graph and the control flow graph of the listing `loop`.

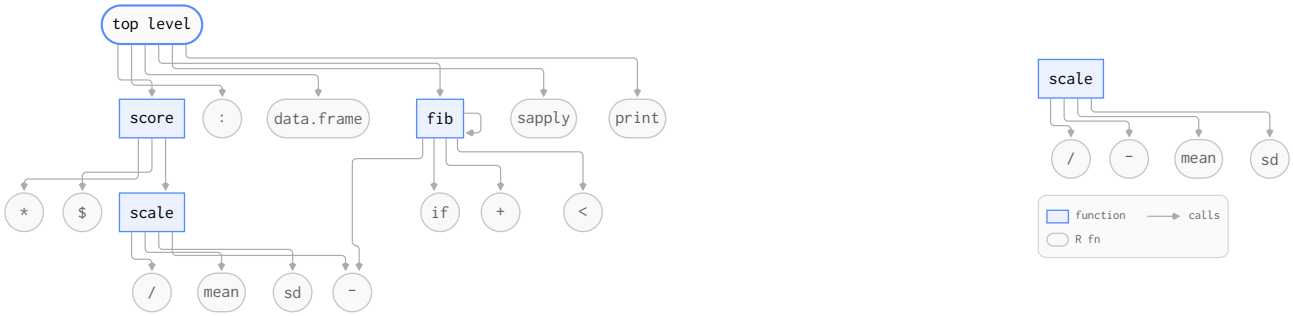
Figure 1 shows the graphs of this listing.

A listing with functions has a call graph:

```
\begin{mr}[mrname=calls]
scale <- function(x) (x - mean(x)) / sd(x)
score <- function(df) scale(df$value) * 10
fib <- function(n) if (n < 2) n else fib(n - 1) + fib(n - 2)
print(score(data.frame(value = sapply(1:8, fib))))
\end{mr}
```

```
scale ⟵ function(x) (x - mean(x)) / sd(x)
score ⟵ function(df) scale(df$value) * 10
fib ⟵ function(n) if (n < 2) n else fib(n - 1) + fib(n - 2)
print(score(data.frame(value = sapply(1:8, fib))))
```

```
\mrCallGraph{calls} \hfill \mrCallGraph[function=scale,legend]{calls}
The \mrDataflowGraph[mermaid=dataflow graph]{calls} and the
\mrCallGraph[mermaid=call graph]{calls} of flowR.
```



The dataflow graph and the call graph of flowR.

Several listings, separated by commas, form one program, as files of a project do. Their dataflow graph frames the vertices of each listing:

```
\begin{mr}[mrname=helpers]
scale <- function(x) (x - mean(x)) / sd(x)
```

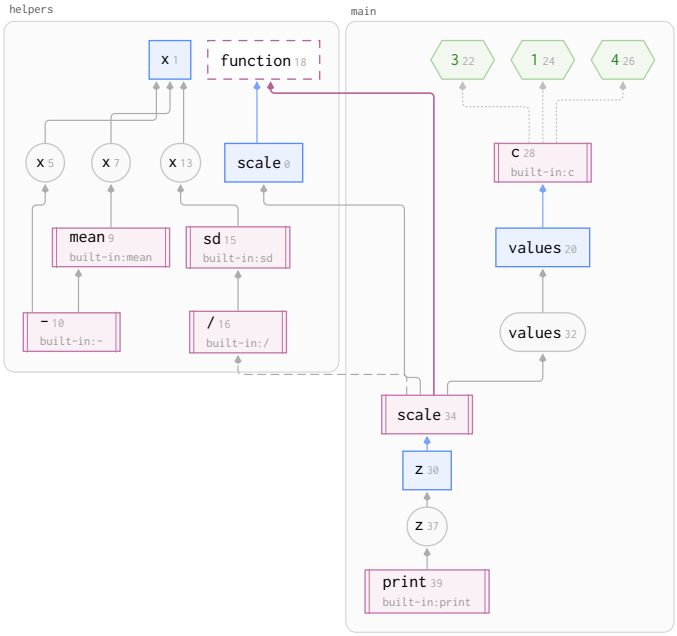
```
\end{mr}
```

```
scale ⟵ function(x) (x - mean(x)) / sd(x)
```

```
\begin{mr}[mrname=main]
values <- c(3, 1, 4)
z <- scale(values)
print(z)
\end{mr}
```

```
values ⟵ c(3, 1, 4)
z ⟵ scale(values)
print(z)
```

```
\mrDataflowGraph{helpers,main}
```



4.9. Opening Listings in flowR's Playground

node docker r web With the option `playground` (or the key `mrplayground` for one listing), a button at the right of the first line opens the code of the listing in flowR's playground, with the criteria of `mrslice` and `mrarrows` marked. The link only needs the script and `node`, not flowR. The code keeps its indentation. This manual has the option on, `mrplayground=false` turns it off for a listing. By default the button is a thin gray ring, `\mrPlayStyle{filled}` (as in this manual, with `mrplay` a blue) a disc. The color `mrplay` and `\mrPlaySize` change it, redefining `\mrPlayButton` replaces it. The button sits `\mrPlayOffset` (`.6em`) right of the text. A negative offset moves it into the text, e.g. `\renewcommand{\mrPlayOffset}{-3em}` left of the badge of `xlistings`. Redefining `\mrPlaceButton{button}` puts it anywhere, and the link moves with it. The playground can start with code the listing does not show. The files of `mrsource` (Section 4) and the listings `mrinclude={name,...}` names (key `mrname`) open as files of their own (`<name>.R`), analyzed first, and `mrprefix={code}` adds code before the listing, one line per statement at each ; outside strings and brackets (`%` and `\#` for `%` and `#`). The marks move down with the code:

```
\begin{minted}[mrinclude=adder,mrprefix={add1 <- adder(1)}]{R}
twice <- function(f, x) f(f(x))
\end{minted}
```

4.10. Requesting flowR Queries

node docker r `\mrFlowrQuery[block=label,path=/json/pointer]{name}{query}` asks flowR any query of its query API about a listing (`block`) or the whole document, and `\mrFlowrValue{name}` shows the part of the answer the JSON pointer `path` names. A query can come before or after its value. For the listing `files` of Section 4.4:

```
\mrFlowrQuery[block=files,path=/dependencies/read/0/value]
  {reads}{[{"type": "dependencies"}]}
\mrFlowrQuery[block=files,path=/dependencies/write/0/functionName]
  {writer}{[{"type": "dependencies"}]}
The listing reads \mrFlowrValue{reads} and writes with \mrFlowrValue{writer}.
```

gives: “The listing reads measurements.csv and writes with write.csv.” Before flowR has answered, queries, graphs, and values show **??**.

5. Computing with R

R With the option `eval`, one `Rscript` answers all queries of a run at its end (`\jobname.mr-eval.R`, answers in `\jobname.mr-eval.out` for the next run), only if they changed. Queries share one environment, in order, like a notebook.

- `\mrValue{expr}` (verbatim) and `\mrValueOf{expr}` (a regular argument): the value as text, 2026 or a, b, c.
- `\mrEval{expr}`, `\mrEvalOf`: what R prints, as code: `[1] 1 9`.
- `\mrYield` in an escape of a listing: the value of its code up to there, as in `print(x) # :yields: !*\mrYield*!` (`:yields:` typesets as →). R runs the code of the listing since its start or the last `\mrYield`, so each line runs once, in the environment of the other queries, and later listings see what earlier ones defined.
- `\mrValueInto\cs{expr}` stores the value, `\mrIfError{then}{else}` tells whether the last query failed, `\mrRTime` and `\mrRTimeText` the time it took.
- `\mrSource{file}` answers the following queries in an environment with the file sourced, `\mrSourceOff` ends that. The environment `mrfile[options]{file}` writes its content to the file, typesets it, and sources it.
- `\mrRSetup{code}` runs code first, `\mrRCommand{Rscript --vanilla}` and `\mrRRun{auto|always|never}` choose how R runs. `\mrEvalSetFormat`, `\mrValueSetFormat`, and `\mrSetErrorFormat` choose how answers look.

```
\begin{mr}
squares <- (1:4)^2
sum(squares) # :yields: !*\mrYield*!
rev(squares) # :yields: !*\mrYield*!
\end{mr}
```

```
squares ⟵ (1:4)^2
sum(squares)  # ⟶ 30
rev(squares)  # ⟶ 16,  9,  4,  1
```

5.1. Showing Console Sessions

The environment `rsession` runs its lines like R’s console (an input may span lines) in an environment of its own, and shows prompts that do not copy. Names defined in a session link to their definition within it. On beamer slides, the inputs appear one after the other (keys `animate`, `start`, `listing`, and `\mrSessionSetup` sets them for all sessions).

```
\begin{rsession}
x <- c(4, 8, 15, 16, 23, 42)
mean(x)
summary(x > 10)
\end{rsession}
```

```
> x ⟵ c(4, 8, 15, 16, 23, 42)
> mean(x)
[1] 18
> summary(x > 10)
   Mode   FALSE    TRUE
logical       2       4
```

5.2. Showing Sessions of flowR's REPL

node docker The environment `flowrsession` runs its lines in flowR's REPL and shows each after the prompt `R>`, with what flowR prints for it: a command (`:query`, `:dataflow*`, `:normalize*`, see `:help`) in the color of keywords, or R. flowR only runs R with the key `rcode` (its engine `r-shell` and `--r-session-access`), else it says that it may not. The keys `animate`, `start`, and `listing` are those of `rsession`. The output is ASCII, as listings reads no other characters (~ for a time, + for a branch of a tree).

```
\begin{flowrsession}
:query [{"type":"linter","rules":["unused-definitions"]}] "x <- 1; y <- x"
\end{flowrsession}
```

```
R> :query [{"type":"linter","rules":["unused-definitions"]}] "x <- 1; y <- x"
Query: linter (21 ms)
   + Unused Definitions (unused-definitions):
       + uncertain:
           + Definition of `y` at 1.9 (1 quick fix(es) available)
       + Metadata: totalConsidered: 2, searchTimeMs: 19, processTimeMs: 2
All queries together required ~21 ms (1ms accuracy, total 23 ms)
```

6. Known Limitations

always Without flowR:

- Names link by the rules of R's search path and the definitions of Section 3.3: a local variable named like a function of a package links to the package, unless the document defines it or `\mrNoLink` keeps it from linking, and a definition that does not start a line (`if (ok) y ⟵ 1`) defines nothing.
- Methods (`setMethod`, `R6Class`) define nothing.
- Backtick names (`` `my var` ``) look like names but only link with flowR.
- Within the argument of another command, inline code needs `\%`, `\#`, and `\$`, and escapes to LaTeX do not work in inline code.

flowR With flowR, these limitations are gone, but

- code that even the lax parser cannot read falls back to the rules without flowR,
- a listing with `mrslicemode=hide` shows all of its code again once, after its code changed,
- a listing with arrows is not broken across pages,
- web only knows packages: no names, lines, slices, graphs, or queries,
- in a control flow graph, the body of a function is a graph of its own, which the calls of the function point to, and flowR has no edge for a `return()` that leaves a function early,
- flowR only sees the text of the escapes of the default `escapeinside={!*}{*!}`. Other escapes (another `escapeinside`, `escapechar`, `mathescape`, `texcl`) work, but marks right of them in the same line can be off.

R R runs all queries of a run in one session. A query that never ends stops all of them.

7. Benchmark

`bench/run.sh` (or `l3build bench`) times the listings of `bench/cases.txt`, generated R code with definitions, pipes, loops, and file access, with pdfTeX's timer (without start-up and shipout). Each case is typeset five times in

four variants, timing the second of two runs: the R of `xlistings` without links, `magic-r` with its database only, with the definitions of the document, and with the answer of flowR. The last column is the whole run that computes the answer of flowR (document, script, and flowR in docker), which a document only pays when its code changed.

Case	Lines	xlistings	magic-r	+ definitions	+ flowR	first run with flowR
a short listing	10	12.2 ms	17.2 ms	18.8 ms	324.1 ms	2 627.5 ms
a page of code	50	21.4 ms	57.1 ms	62.3 ms	1 483.9 ms	8 252.7 ms
four pages of code	200	71.5 ms	226.2 ms	231.0 ms	6 021.2 ms	31 226.9 ms

A. The Shipped Database

The table lists each package with its version, its documentation ([R] R's manual, [C] rdrr.io), and the number of names that link.

`generate` builds the database from `tools/packages.txt`: names from flowR's signature database, help topics from R's help index, versions from the installed packages. It requests every linked page, and a name whose page fails moves to the first page that works (the page of its own name, then search.r-project.org) or is dropped. `verify` requests every page of a database and lists the ones that fail (rdrr.io can lag behind CRAN), `--prune` moves or drops their names as `generate` does. A weekly job of the repository verifies the database.

```
node magic-r-flowr.mjs generate    # writes magic-r-links.cfg from tools/packages.txt
node magic-r-flowr.mjs verify      # lists the pages that fail
node magic-r-flowr.mjs verify --prune # ... and moves or drops their names
```

package	version	names	package	version	names	package	version	names
base [R]	4.6.1	1285	methods [R]	4.6.1	200	datasets [R]	4.6.1	108
utils [R]	4.6.1	340	grDevices [R]	4.6.1	157	graphics [R]	4.6.1	117
stats [R]	4.6.1	735	tools [R]	4.6.1	148	parallel [R]	4.6.1	38
grid [R]	4.6.1	264	splines [R]	4.6.1	20	stats4 [R]	4.6.1	1
compiler [R]	4.6.1	9	tcltk [R]	4.6.1	280	MASS [C]	7.3-65	280
Matrix [C]	1.7-5	174	lattice [C]	0.22-9	236	survival [C]	3.8-6	311
nlme [C]	3.1-169	541	boot [C]	1.3-32	86	cluster [C]	2.1.8.2	45
codetools [C]	0.2-20	19	foreign [C]	0.8-91	17	KernSmooth [C]	2.23-26	7
rpart [C]	4.1.27	31	class [C]	7.3-23	17	nnet [C]	7.3-20	25
spatial [C]	7.3-18	25	mgcv [C]	1.9-4	234	tidyverse [C]	2.0.0	8
broom [C]	1.0.12	258	conflicted [C]	1.2.0	5	cli [C]	3.6.6	248
dbplyr [C]	2.5.2	417	dplyr [C]	1.2.1	411	dtplyr [C]	1.3.3	30
forcats [C]	1.0.1	42	ggplot2 [C]	4.0.3	726	googledrive [C]	2.1.2	124
googlesheets4 [C]	1.1.2	72	haven [C]	2.5.5	47	hms [C]	1.1.4	46
httr [C]	1.4.8	109	jsonlite [C]	2.0.0	23	lubridate [C]	1.9.5	174
magrittr [C]	2.0.5	42	modelr [C]	0.1.11	44	pillar [C]	1.11.1	103
purrr [C]	1.2.2	189	ragg [C]	1.5.2	9	readr [C]	2.2.0	113
readxl [C]	1.4.5	9	reprex [C]	2.1.1	13	rlang [C]	1.2.0	456
rstudioapi [C]	0.19.0	140	rvest [C]	1.0.5	59	stringr [C]	1.6.0	62
tibble [C]	3.3.1	60	tidyr [C]	1.3.2	127	xml2 [C]	1.5.2	152
tidyselect [C]	1.2.1	31	glue [C]	1.8.1	21	withr [C]	3.0.2	79
vctrs [C]	0.7.3	399	lifecycle [C]	1.0.5	16	scales [C]	1.4.0	250
fs [C]	2.1.0	91	here [C]	1.0.2	4	R6 [C]	2.6.1	7
patchwork [C]	1.3.2	42	data.table [C]	1.18.4	218	yaml [C]	2.3.12	6
httr2 [C]	1.2.2	152	curl [C]	7.1.0	46	writexl [C]	1.5.4	4
DBI [C]	1.3.0	55	RSQLite [C]	2.4.5	39	testthat [C]	3.3.2	218
devtools [C]	2.5.0	74	usethis [C]	3.2.1	178	roxygen2 [C]	8.0.0	176
knitr [C]	1.51	146	rmarkdown [C]	2.31	95	shiny [C]	1.14.0	306
Rcpp [C]	1.1.1-1.1	27	lme4 [C]	2.0.1	152	caret [C]	7.0.1	365
randomForest [C]	4.7.1-2	38	car [C]	3.1.5	360	zoo [C]	1.8.15	313
xts [C]	0.14.2	108	future [C]	1.70.0	169	furrr [C]	0.4.0	59
plotly [C]	4.12.0	164	igraph [C]	2.3.3	891	units [C]	1.0.1	89
janitor [C]	2.2.1	52	openxlsx [C]	4.2.8-1	87	sf [C]	1.1.0	502

package	version	names	package	version	names	package	version	names
emmeans [C]	2.0.2	159	gt [C]	1.3.0	230			

Index of Commands, Options, and Keys

animate (key) R, 13
arrowroute (package option) flowR, 3
attach (key), 3

block (key) flowR, 13
\bmr, 2, 5

cleanup (package option) flowR, 3

database (package option), 2
dependencies (package option) flowR, 3

eval (package option) R, 3

flowr (package option) flowR, 2
flowr (key) flowR, 7, 10
flowr-backend (package option) flowR, 2
flowr-command (package option) flowR, 2
flowr-engine (package option) flowR, 2
flowr-image (package option) flowR, 2
flowr-required (package option) flowR, 2
flowr-version (package option) flowR, 2
flowrsession (environment) flowR, 14
function (key) flowR, 10

graphs (package option) flowR, 3
graphs-gray (package option) flowR, 3
graphs-legend (package option) flowR, 3
graphs-lines (package option) flowR, 3

index (package option), 2
interactive (package option) flowR, 3

language (package option), 2
legend (key) flowR, 10
ligatures (package option), 2
lines (key) flowR, 10
links (package option), 2
lint (package option) flowR, 2
listing (key) R, 13
locallinks (package option), 2

mermaid (key) flowR, 10
mr (environment), 2
\mrAddLink, 5
\mrArrowGap flowR, 9
\mrArrowLane flowR, 9
mrarrowroute (key) flowR, 9
mrarrows (key) flowR, 9
\mrArrowTrack flowR, 9
\mrAttach, 3
\mrCallGraph flowR, 10
mrcallsarrows (key) flowR, 9
mrcapture (key) flowR, 6
\mrControlFlowGraph flowR, 10
\mrCran, 3
\mrDataflowGraph flowR, 10
\mrDefaultSearchPath, 3
mrdefinitions (key), 4
mrdependencies (key) flowR, 7
\mrDetach, 3
\mrDocsCran, 3
\mrDocsRManual, 3
\mrEval R, 13
\mrEvalOf R, 13
\mrEvalSetFormat R, 13
mrfile (environment) R, 13
mrflowr (key) flowR, 7
\mrFlowrDescription flowR, 7
\mrFlowrOptions flowR, 7
\mrFlowrQuery flowR, 13
\mrFlowrRun flowR, 7
\mrFlowrValue flowR, 13
\mrGraphBuiltin flowR, 10
\mrGraphId flowR, 10
\mrGraphLine flowR, 10
\mrIfError R, 13
mrinclude (key) flowR, 12
mrindex (key), 5
mrinteractive (key) flowR, 9
mrlibrary (key), 3
\mrLigature, 6
\mrLigatureKern, 5
\mrLigaturesOff, 2
\mrLigaturesOn, 2
mrlinks (key), 2
\mrLinksOff, 2
\mrLinksOn, 2
mrlint (key) flowR, 8
\mrLinterRules flowR, 8
\mrLintShape flowR, 8
\mrLoadPackage, 3
mrlocallinks (key), 2
\mrLocalLinksOff, 2
\mrLocalLinksOn, 2
mrname (key), 5, 10
\mrNodeCommand flowR, 7
\mrNoLink, 5, 14
\mrOperators, 5
\mrPackage, 5
mrpackage (key), 3
\mrPlaceButton flowR, 12
\mrPlayButton flowR, 12
mrplayground (key) flowR, 12

\mrPlayOffset flowR, 12
\mrPlaySize flowR, 12
\mrPlayStyle flowR, 12
mrprefix (key) flowR, 12
\mrRCommand R, 13
\mrRRun R, 13
\mrRSetup R, 13
\mrRTime R, 13
\mrRTimeText R, 13
\mrScope flowR, 6
mrscope (key) flowR, 6
\mrSearchPath, 3
\mrSessionSetup R, 13
\mrSetErrorFormat R, 13
\mrSetStyle, 4
mrslice (key) flowR, 8
mrslicemode (key) flowR, 8
\mrSource R, 13
mrsource (key) flowR, 6
\mrSourceOff R, 13
\mrTightLigature, 6
\mrTightLigatures, 5
\mrTightOff, 2
\mrTightOn, 2
\mrUnderlineOff, 2
\mrUnderlineOn, 2
\mrUse, 5
\mrUseAt, 5
\mrValue R, 13
\mrValueInto R, 13
\mrValueOf R, 13
\mrValueSetFormat R, 13
\mrVersion, 3
\mrYield R, 13

operators (package option), 2

path (key) flowR, 13
playground (package option) flowR, 3

rcode (key) flowR, 14
rowsep (key) flowR, 10
rsession (environment) R, 13, 14

slices (package option) flowR, 2
start (key) R, 13
strict (package option), 2
symbols (package option), 2

tight (package option), 2

underline (package option), 2

xlistings (package option), 3